

Pearls AQI Predictor

Air Quality Forecasting System for Karachi

Submitted by	Izaan Omair
Role	Data Science Intern
Organization	10Pearls
City	Karachi, Pakistan
Submitted	June 06, 2026

Contents

1. Background and Motivation
2. What the System Does
3. How the Pipeline Works
4. The Data
5. Model Training and Results
6. The Dashboard
7. Automation
8. Problems Encountered and How I Solved Them
9. How to Run It
10. What Was Delivered
11. What I Would Do Differently
12. Final Thoughts

1. Background and Motivation

Karachi is among the most polluted major cities in Asia. On bad days, the Air Quality Index regularly crosses 150 — the threshold where outdoor activity becomes genuinely harmful, especially for children, the elderly, and anyone with respiratory conditions. Despite this, most people in the city have no way of knowing in advance how bad the air will be tomorrow.

This project started with a simple question: can we predict, using only free public data, what Karachi's AQI will look like 24 hours from now? Not with expensive sensor arrays or proprietary datasets — just the AQICN air quality API, the OpenWeatherMap weather API, and a standard machine learning pipeline.

The answer turned out to be yes — though getting there required solving a number of real technical problems along the way, which made the project significantly more instructive than if everything had worked from the start.

2. What the System Does

At a high level, the system does four things continuously and automatically:

- Every hour, it fetches the current AQI readings and weather conditions for Karachi from two free APIs, engineers a set of features from that data, and stores it locally.
- Every day, it retrains three machine learning models on the accumulated data, producing 24-hour AQI forecasts and calculating SHAP values to explain which features drove each prediction.
- At any time, a user can open the dashboard and see the current AQI, the predicted AQI for the next 24, 48, and 72 hours, a historical trend chart, pollutant breakdowns, and a breakdown of which features the model relied on most.
- The entire thing runs for free — the APIs are free, the compute is GitHub Actions' free tier, the hosting is Streamlit Cloud's free tier, and the database is SQLite built into Python.

3. How the Pipeline Works

The project is split into separate modules, each responsible for one part of the process. This made debugging much easier — when something broke, I could isolate which stage had the problem without touching the rest.

Module	What It Does
feature_pipeline/fetch.py	Calls AQICN and OpenWeatherMap APIs and returns raw readings

Module	What It Does
feature_pipeline/engineer.py	Turns raw readings into 40+ ML features (lags, rolling stats, time encodings, derived ratios)
feature_pipeline/store.py	Saves features to a local SQLite database for training
synthetic_backfill.py	Generates a full year of synthetic hourly data for initial training — no API required
training_pipeline/train.py	Trains Random Forest, Ridge Regression, and XGBoost on stored features
inference_pipeline/predict.py	Loads trained models and returns 24/48/72-hour AQI forecasts with SHAP values
dashboard/app.py	Streamlit dashboard — displays everything in a live, interactive interface
api/app.py	Optional FastAPI endpoint for accessing forecasts programmatically

Table 1: Pipeline modules and their responsibilities

4. The Data

Where it comes from

The system uses two data sources, both free with instant signup:

- AQICN (aqicn.org) — provides real-time AQI, PM2.5, PM10, ozone, nitrogen dioxide, sulfur dioxide, and carbon monoxide readings for Karachi.
- OpenWeatherMap — provides temperature, humidity, wind speed, atmospheric pressure, and cloud cover. These matter because weather conditions drive pollution dispersion (wind carries pollutants away; humidity and temperature affect chemical reactions in the air).

Feature engineering

Raw API readings alone are not very useful for a machine learning model. The real work is in creating features that capture patterns the model can learn from. I engineered over 40 features across six categories:

Category	Examples	Why they matter
Time	hour, day_of_week, month, is_weekend, sine/cosine encodings	AQI follows strong daily and weekly cycles — rush hour, weekday vs weekend

Category	Examples	Why they matter
Pollutants	Current aqi, pm25, pm10, o3, no2, so2, co	Baseline readings the model uses to understand current state
Weather	temperature, humidity, wind_speed, pressure	Wind disperses pollution; humidity and temperature affect chemistry
Lag features	aqi_lag_1h, 3h, 6h, 12h, 24h; pm25 lags	Recent history is the strongest predictor of near-future AQI
Rolling stats	roll_mean_6h/12h/24h, roll_std_6h, roll_max_24h	Captures whether pollution is trending up, stable, or volatile
Derived	aqi_change_rate, pm25_pm10_ratio, pollution_load, wind_aqi_ratio	Interaction terms that capture relationships between variables

Table 2: Feature categories, examples, and rationale

The training data problem — and how I fixed it

This was the central technical challenge of the project. When I first tried to train models, all three produced negative R^2 scores — meaning they were literally worse than guessing the average AQI every time. After investigating, I found two problems:

- The database only had 96 rows — one per day for about three months. That is nowhere near enough data for the lag features to work properly. aqi_lag_1h on daily data represents yesterday's AQI, not one hour ago. The model was learning from nonsense.
- Even after fixing the frequency, training on 96 samples means the model overfits to random noise rather than learning real patterns.

The fix was to write a standalone backfill script (synthetic_backfill.py) that generates a full year of continuous hourly records — 8,760 rows — using realistic seasonal variation, daily peak patterns, and random noise anchored to Karachi's real AQI baseline. No API required. This runs automatically in GitHub Actions before every training job.

5. Model Training and Results

Training setup

I trained three models to predict AQI 24 hours ahead. The data was split 80/20 chronologically — the first 80% for training, the last 20% for testing. Shuffling was deliberately disabled, because shuffling a time series would let the model see future data during training, which would produce misleadingly good test scores.

Results on live data

The numbers below are from the live dashboard — real Karachi AQI data, not the synthetic training set:

Model	R ²	RMSE	MAE
Ridge Regression	0.274	8.408	6.183
Random Forest	0.262	8.479	6.392
XGBoost	0.235	8.631	6.536

Table 3: Model performance on live production data

What these numbers mean

An R² of 0.274 means the model explains about 27% of the variance in AQI 24 hours ahead. That sounds modest, but consider what it is working with: a single city, only free public data, and a 24-hour forecast horizon. For context, predicting tomorrow's weather from today's readings is hard even with dedicated meteorological infrastructure.

An RMSE of 8.4 AQI units means the average prediction is off by about 8 points. On a scale where the difference between Good (0-50) and Moderate (51-100) is 50 points, that is quite precise. In practical terms, if the model predicts AQI 80, the real value will typically be somewhere between 72 and 88.

Before and after the backfill fix

Model	R ² before fix (96 rows)	R ² after backfill (8,760 rows)	R ² on live data
Ridge Regression	-0.106	+0.476	0.274
Random Forest	-0.174	+0.447	0.262
XGBoost	-0.017	+0.409	0.235

Table 4: R² before fix, after backfill training, and on live data

The gap between post-backfill R² (around 0.45) and live R² (around 0.26) is normal and expected. Synthetic data has controlled variation; live data has unpredictable pollution events, unusual weather, and real-world noise. The fact that R² is still positive on live data means the model genuinely learned something useful — it is not just memorizing training examples.

6. The Dashboard

The Streamlit dashboard is the main user-facing part of the project. It runs locally at <http://localhost:8501> and is also deployed publicly on Streamlit Cloud. Here is what it shows:

- A live AQI reading for Karachi with color-coded severity (green through red)
- Current pollutant readings — PM2.5, PM10, ozone, NO2, SO2, CO — with bar indicators
- 72-hour forecast cards showing predicted AQI at +24h, +48h, and +72h
- A 48-hour historical AQI chart with the forecast plotted as a continuation
- SHAP feature importance chart showing which inputs most influenced the prediction
- Model performance metrics (R^2 , RMSE, MAE) for all three models side by side
- Automatic warning banners when AQI crosses 100 or 150
- An optional auto-refresh toggle that updates the data every 60 seconds

The dashboard was also a source of several bugs. The original version used hardcoded dark background colors in the CSS, which made all text invisible when Streamlit was set to light mode. I fixed this by removing the forced background colors and using transparent Plotly chart backgrounds so the theme adapts automatically.

7. Automation

The whole pipeline runs automatically using GitHub Actions, within the free tier. Two workflows handle everything:

Workflow	When it runs	What it does
Feature Pipeline	Every hour	Fetches live AQI and weather data, engineers features, saves to SQLite
Training Pipeline	2:00 AM UTC daily	Runs synthetic backfill, trains all three models, saves model files as artifacts

Table 5: GitHub Actions workflows

Getting the automation to work took several iterations. The original workflow used `STORAGE_MODE=hopsworks`, which meant it tried to connect to a cloud database that did not exist on the GitHub runner, and the training job saw zero rows every time. Switching to local SQLite and adding the synthetic backfill as a step before training resolved this completely.

8. Problems Encountered and How I Solved Them

Most of what I learned on this project came from things breaking. Here is an honest account of the main issues and what fixed them:

Problem	What was actually wrong	How I fixed it
All models had negative R^2	Only 96 daily rows. Lag features computed on row index — so <code>aqi_lag_1h</code> meant yesterday, not one hour ago	Wrote <code>synthetic_backfill.py</code> to generate 8,760 continuous hourly rows with proper timestamps
GitHub Actions: 0 rows after backfill	The original backfill called the AQICN API, which failed silently on the runner because the token was invalid. It returned 0 rows without crashing	Replaced API-based backfill with pure-math synthetic generation requiring no network call
Training still saw 0 rows after API fix	<code>config.py</code> auto-upgraded <code>STORAGE_MODE</code> to <code>hopsworks</code> if any <code>Hopsworks</code> key was present, overriding the local setting	Removed the auto-upgrade logic from <code>config.py</code>
TensorFlow would not install	TensorFlow dropped Windows support after v2.10 and does not support Python 3.13+	Wrapped LSTM model loading in <code>try/except</code> so the system works fine without it
<code>pip install -r requirements.txt</code> failed	<code>hopsworks</code> package requires the <code>twofish</code> library which needs C++ Build Tools to compile	Bypassed <code>requirements.txt</code> and installed only the packages actually needed
streamlit not recognized as a command	Windows PATH does not expose venv scripts when using bare commands	Used <code>python -m streamlit run</code> instead of <code>streamlit run</code> throughout
Virtual environment used Python 3.14	<code>py</code> command defaulted to latest installed Python even with older versions available	Used the full path <code>C:\Python311\python.exe -m venv venv</code> to force the right version
Streamlit Cloud used Python 3.14	No <code>.python-version</code> file in the repo; Streamlit Cloud picked the latest Python	Added a <code>.python-version</code> file containing '3.11' to the repo root
Dashboard invisible in light mode	CSS had hardcoded dark backgrounds; text was white-on-white in light mode	Removed forced background colors; changed Plotly backgrounds to transparent
Plotly titlefont error	The <code>titlefont</code> property was deprecated in newer Plotly versions	Replaced <code>titlefont=dict(...)</code> with <code>title=dict(text=..., font=dict(...))</code>

Table 6: Issues encountered during development and their resolutions

9. How to Run It

The project runs on Windows, Mac, or Linux with Python 3.10, 3.11, or 3.12. Python 3.13 and 3.14 are not supported by several key dependencies.

Step	Command
Check available Python versions	<code>py --list</code>
Find path to Python 3.11	<code>py -3.11 -c "import sys; print(sys.executable)"</code>
Create virtual environment	<code>C:\Python311\python.exe -m venv venv</code>
Activate it	<code>venv\Scripts\activate</code>
Verify Python version	<code>python --version</code> (must show 3.11.x)
Install packages	<code>pip install requests loguru streamlit fastapi uvicorn scikit-learn xgboost joblib pandas numpy python-dotenv plotly shap</code>
Set up .env file	<code>AQICN_TOKEN=... OW_API_KEY=... TARGET_CITY=karachi STORAGE_MODE=local</code>
Generate training data	<code>python synthetic_backfill.py karachi 365</code>
Train the models	<code>python -m training_pipeline.train</code>
Launch the dashboard	<code>python -m streamlit run dashboard/app.py</code>

Table 7: Setup steps

After the first setup, the only commands needed daily are activating the venv and running the dashboard. The GitHub Actions workflows handle data collection and retraining automatically in the background.

10. What Was Delivered

Item	Status
End-to-end ML pipeline from data ingestion to live dashboard	Done
40+ engineered features across six categories	Done
Three trained models — Random Forest, Ridge, XGBoost	Done
Synthetic backfill script that runs without any API credentials	Done

Item	Status
Streamlit dashboard with live readings, forecasts, and SHAP charts	Done
SHAP feature importance for model explainability	Done
FastAPI endpoint for programmatic access to forecasts	Done
GitHub Actions — hourly feature pipeline	Running
GitHub Actions — daily model retraining	Running
Streamlit Cloud deployment with public URL	Live
README with full setup guide and troubleshooting section	Done
This project report	Done

Table 8: Deliverables and status

11. What I Would Do Differently

Looking back, a few things stand out as areas where a different approach would have saved time or produced better results:

- Start collecting real hourly data on day one. The synthetic backfill solved the immediate problem, but real data collected over a few months would produce substantially better models. I would set up the hourly feature pipeline as the very first step and let it run while building everything else.
- Validate feature computation against timestamps earlier. The lag feature bug — where `aqi_lag_1h` represented yesterday's value rather than one hour ago — took a while to diagnose. A simple sanity check comparing computed lag values against actual historical readings would have caught this immediately.
- Lock the Python version in requirements from the start. Most of the dependency problems (TensorFlow, twofish, Streamlit Cloud using 3.14) came from not specifying a Python version constraint upfront. A `.python-version` file and a clean `requirements.txt` would have avoided several hours of debugging.
- Add wind direction and rainfall as features. Wind speed matters, but wind direction matters more — pollution from industrial areas to the west of Karachi has a very different impact depending on which way the wind is blowing.
- Run a proper hyperparameter search on XGBoost and Random Forest rather than using reasonable defaults. Even a simple grid search would likely improve R^2 by a few percentage points.

12. Final Thoughts

When I started this project, all three models had negative R^2 . That meant every prediction was worse than just guessing the average AQI. Understanding why that happened — tracing it back to 96 rows of daily data and incorrectly computed lag features — and then fixing it was genuinely the most useful thing I did on this project. Good models need good data, and good data needs to be structured correctly.

The technical problems around Python versions, dependency conflicts, and GitHub Actions configuration were frustrating in the moment but worth working through. They are exactly the kinds of problems that come up in real data science work, and knowing how to debug them systematically is as important as knowing how to train a model.

The final system — a live dashboard forecasting Karachi's air quality 24 hours ahead, retrained automatically every day, accessible from any browser for free — is something I am genuinely proud of. The R^2 of 0.27 is honest: it reflects what is actually achievable with free data and a few months of development. With a year of real hourly data, I would expect that number to improve significantly.